



28. Python: Graphical User Interface with Tkinter

"Turning Python programs into applications people can actually click."

Previous Content:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 27. Python: Drawing with Turtle Graphics](#)

Contents:

[1. Some Tkinter Keywords](#)

[2. Basic Operations on Tkinter](#)

[2.1 Adding a Widget \(Displaying Text\)](#)

[2.2. Label and Frame](#)

[2.3. Entry and Text](#)

[2.4. Buttons and Selection Widgets](#)

[2.5. User Input With Text Widgets](#)

[2.6. Place, Pack and Grid](#)

[2.7. Canvas and PhotoImage](#)

[2.8. Configuration and Tables](#)

[2.9. Pop-Up Windows](#)

[2.10. Message Boxes](#)

[2.11. Colors and Sizes](#)

[2.12. Read-Only Entries](#)

[3. Some Basic Systems with Tkinter](#)

[3.1. Student Information Table](#)

[3.2. Simple Text Editor](#)

[3.3. Drawing Pad](#)

[3.4. Simple Calculator](#)

[3.5. Tic-Tac-Toe Game](#)

[3.6. Other Systems Built with Tkinter](#)

[Summary](#)

[References](#)

Up until now, most of our Python programs have lived inside the terminal. The user enters some text. The program performs a calculation. The result appears as more text.

And while there's absolutely nothing wrong with that, most applications people use every day don't look like a terminal.

They have: Windows; Buttons; Menus; Text boxes; Images; Dialog boxes

In other words, they have a **Graphical User Interface (GUI)**. A GUI allows users to interact with a program visually instead of typing commands into a console.

Think about some of the software you use regularly: Web browsers; Calculators; Music players; Chat applications; Text editors. Almost all of them are GUI applications.

Fortunately, Python provides a built-in library called **Tkinter** that allows us to create these interfaces with surprisingly little code. With Tkinter, we can create: Windows; Labels; Buttons; Text fields; Checkboxes; Menus; Dialog boxes. And much more.

Now, to be fair, Tkinter is not the most modern GUI framework available today.

Frameworks such as: PyQt; PySide; Kivy; wxPython; or HTML/CSS/JavaScript provide more advanced features and modern appearances.

However, Tkinter has one enormous advantage: **It comes with Python**. No extra installation. No complicated setup. Just import the library and start building. And for learning GUI development, that simplicity makes Tkinter one of the best places to begin.

Before we start creating windows and applications, let's first become familiar with some of the most important Tkinter components.

1. Some Tkinter Keywords

Below are some of the most commonly used classes and methods in Tkinter.

Keyword / Method	Description
<code>Tk()</code>	Creates the main application window
<code>mainloop()</code>	Runs the application event loop
<code>title()</code>	Sets the window title
<code>geometry()</code>	Sets the window size
<code>Label()</code>	Displays text or images
<code>Button()</code>	Creates a clickable button
<code>Entry()</code>	Creates a single-line text field
<code>Text()</code>	Creates a multi-line text area
<code>Frame()</code>	Creates a container for widgets
<code>Checkbutton()</code>	Creates a checkbox
<code>Radiobutton()</code>	Creates a radio button
<code>Listbox()</code>	Creates a selectable list
<code>Menu()</code>	Creates menu bars and menus
<code>Canvas()</code>	Creates a drawing area
<code>PhotoImage()</code>	Displays image files
<code>pack()</code>	Places widgets automatically
<code>grid()</code>	Places widgets in rows and columns
<code>place()</code>	Places widgets at exact coordinates
<code>config()</code>	Modifies widget properties
<code>destroy()</code>	Closes a widget or window
<code>get()</code>	Retrieves user input
<code>insert()</code>	Inserts text into a widget
<code>delete()</code>	Removes content from a widget
<code>messagebox.showinfo()</code>	Displays an information dialog
<code>messagebox.showwarning()</code>	Displays a warning dialog
<code>messagebox.showerror()</code>	Displays an error dialog

Most Tkinter applications are built from these few components. A typical program usually follows a simple pattern:

1. Create a window.
2. Create widgets.
3. Position the widgets.
4. Define what happens when users interact with them.
5. Start the application using `mainloop()`.

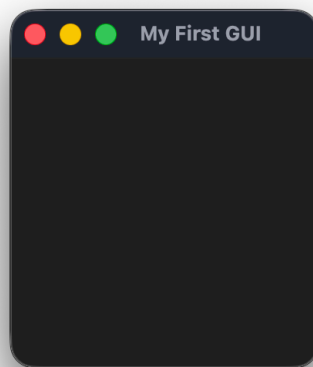
For example, the smallest possible Tkinter program looks like this:

```
import tkinter as tk

window = tk.Tk()
window.title("My First GUI")

window.mainloop()
```

When this program runs, Python creates a blank application window and waits for user interaction.



It may not look impressive yet. But that tiny window is the foundation of every Tkinter application we will build in this chapter.

2. Basic Operations on Tkinter

Now that we've seen some of the most important Tkinter components, it's time to actually use them. And here's something you'll quickly discover: Most GUI

applications are built from surprisingly simple pieces.

- A button is just a button.
- A text field is just a place to type.
- A window is simply a container that holds everything else.

The complexity comes from how we combine these pieces together. Think about a calculator application. At its core, it only needs a window, a display area, and a few buttons. That's it. The same idea applies to text editors, login systems, and even larger desktop applications.

So instead of trying to learn everything at once, let's build our understanding one widget at a time.

2.1 Adding a Widget (Displaying Text)

The first thing most GUI applications display is text. Tkinter provides the `Label()` widget for this purpose. A label can display messages, instructions, titles, results

Unlike an Entry widget, a Label is not editable by the user.

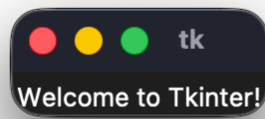
```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="Welcome to Tkinter!")
label.pack()

root.mainloop()
```

When the program runs, the text appears inside the application window.



Simple. But almost every Tkinter application starts with labels somewhere.

2.2. Label and Frame

As applications become larger, organizing widgets becomes important. This is where Frames help. A Frame acts like a container that groups related widgets together. Think of it as creating sections inside your application.

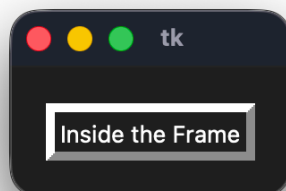
```
import tkinter as tk

root = tk.Tk()

frame = tk.Frame(root, borderwidth=5, relief=tk.RAISED)
frame.pack(padx=20, pady=20)

label = tk.Label(frame, text="Inside the Frame")
label.pack()

root.mainloop()
```



Frames can have different border appearances.

Style	Description
<code>tk.FLAT</code>	No border effect (default)
<code>tk.SUNKEN</code>	Creates a sunken appearance
<code>tk.RAISED</code>	Creates a raised appearance
<code>tk.GROOVE</code>	Creates a grooved border
<code>tk.RIDGE</code>	Creates a ridged border

These styles are purely visual, but they help users understand how different sections of a program are organized.

A `LabelFrame` is a special type of `Frame` that automatically displays a title around its border. It is commonly used to group related widgets together, making larger applications easier to organize and understand.

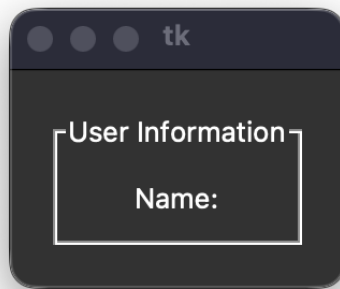
```
import tkinter as tk

root = tk.Tk()

frame = tk.LabelFrame(root, text="User Information",
                      padx=10, pady=10)
frame.pack(padx=20, pady=20)

tk.Label(frame, text="Name:").pack()

root.mainloop()
```



Unlike a normal Frame, a LabelFrame provides a visible caption, allowing users to immediately understand the purpose of the widgets contained inside it.

This keeps it short while naturally introducing a widget that students will frequently encounter in forms, settings panels, and login screens.

2.3. Entry and Text

Labels display information. But what if we want the user to provide information? That's where Entry and Text widgets come in.

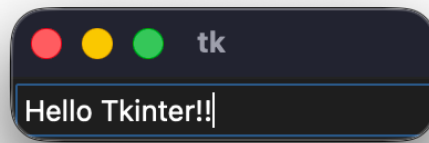
- Entry is used for a single line of input.

```
import tkinter as tk

root = tk.Tk()

entry = tk.Entry(root)
entry.pack()

root.mainloop()
```

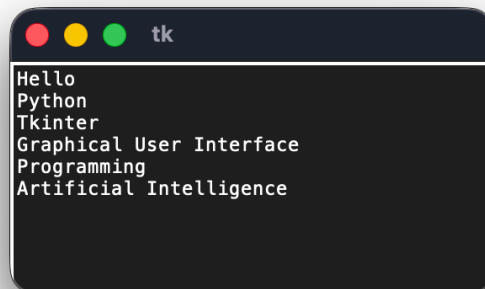
- Text is used for multiple lines of text.

```
import tkinter as tk

root = tk.Tk()

textbox = tk.Text(root, width=40, height=10)
textbox.pack()

root.mainloop()
```



If Entry is like a single sentence, then Text is like an entire document.

2.4. Buttons and Selection Widgets

Buttons allow users to trigger actions. Without buttons, applications would be difficult to interact with.

- Normal Button

```
import tkinter as tk

root = tk.Tk()

button = tk.Button(root, text="Click Me")
button.pack()

root.mainloop()
```



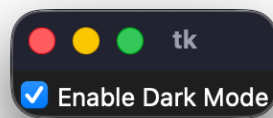
- Check button is used when multiple choices are allowed.

```
import tkinter as tk

root = tk.Tk()

check = tk.Checkbutton(root, text="Enable Dark Mode")
check.pack()

root.mainloop()
```



- Listbox is used for displaying multiple selectable items.

```
import tkinter as tk

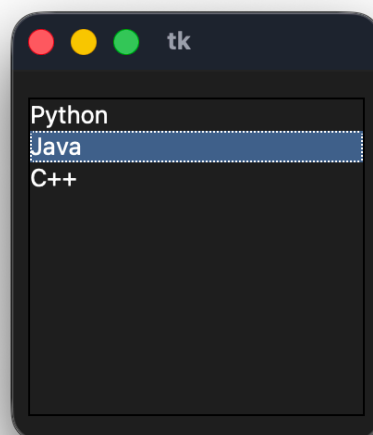
root = tk.Tk()

listbox = tk.Listbox(root)

listbox.insert(1, "Python")
listbox.insert(2, "Java")
listbox.insert(3, "C++")

listbox.pack()

root.mainloop()
```



- Option Menu is used for drop-down selections.

```
import tkinter as tk

root = tk.Tk()

choice = tk.StringVar(root)
choice.set("Python")

menu = tk.OptionMenu(root, choice, "Python", "Java", "C++")
menu.pack()

root.mainloop()
```

Selection widgets help users choose information instead of typing everything manually.



2.5. User Input With Text Widgets

Displaying widgets is nice. But real applications need to process user input. Tkinter provides several useful methods for this purpose.

Common Methods

Method	Purpose
<code>get()</code>	Retrieve widget content
<code>delete()</code>	Remove content
<code>insert()</code>	Insert content
<code>selection()</code>	Get selected item

Method	Purpose
<code>destroy()</code>	Remove widget/window
<code>get_children()</code>	Retrieve child widgets

- Let's create a Decimal → Binary → Hexadecimal converter.

```
import tkinter as tk

def convert():
    value = int(entry.get())
    binary_label.config(text=f"Binary: {bin(value)}")
    hex_label.config(text=f"Hex: {hex(value)}")

root = tk.Tk()

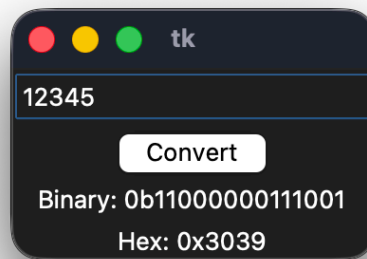
entry = tk.Entry(root)
entry.pack()

button = tk.Button(root, text="Convert", command=convert)
button.pack()

binary_label = tk.Label(root, text="Binary:")
binary_label.pack()

hex_label = tk.Label(root, text="Hex:")
hex_label.pack()

root.mainloop()
```



Notice how the user enters a number → `get()` retrieves the value → Python converts it → The labels display the result.

This interaction is the foundation of GUI programming.

2.6. Place, Pack and Grid

Creating widgets is only half the story. We must also decide where they appear. Tkinter provides three layout managers.

- `label.pack()` : automatically arranges widgets
- `label.grid(row=0, column=0, padx=12, pady=15)` : uses rows and columns.
- `label.place(x=100, y=50)` : **Place** uses exact coordinates.

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Layout Managers Example")

# Using pack()
frame_pack = ttk.LabelFrame(root, text="Pack (Top-to-Bottom)")
frame_pack.pack(side="left", padx=10, pady=10, fill="both",
expand=True)

ttk.Button(frame_pack, text="Button 1").pack(pady=5)
```

```

ttk.Button(frame_pack, text="Button 2").pack(pady=5)

# Using grid()
frame_grid = ttk.LabelFrame(root, text="Grid (Rows & Column
s)")
frame_grid.pack(side="left", padx=10, pady=10, fill="both",
expand=True)

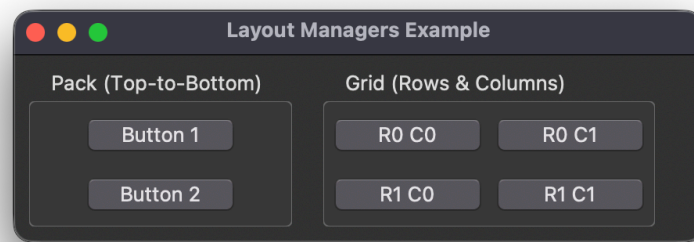
ttk.Button(frame_grid, text="R0 C0").grid(row=0, column=0,
padx=5, pady=5)
ttk.Button(frame_grid, text="R0 C1").grid(row=0, column=1,
padx=5, pady=5)
ttk.Button(frame_grid, text="R1 C0").grid(row=1, column=0,
padx=5, pady=5)
ttk.Button(frame_grid, text="R1 C1").grid(row=1, column=1,
padx=5, pady=5)

# Using place()
frame_place = ttk.LabelFrame(root, text="Place (Exact Coord
inates)")
frame_place.pack(side="left", padx=10, pady=10, fill="bot
h", expand=True)

ttk.Button(frame_place, text="X=20, Y=20").place(x=20, y=2
0)
ttk.Button(frame_place, text="X=50, Y=70").place(x=50, y=7
0)

root.mainloop()

```



2.7. Canvas and PhotoImage

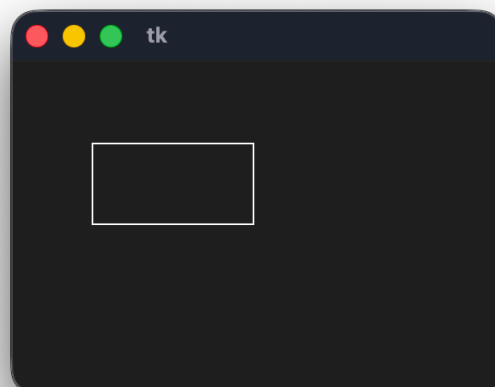
Sometimes we need graphics instead of text. Tkinter provides the Canvas widget.

```
import tkinter as tk

root = tk.Tk()

canvas = tk.Canvas(root, width=300, height=200)
canvas.pack()
canvas.create_rectangle(50, 50, 150, 100)

root.mainloop()
```



Canvas can draw lines, circles, rectangles, text, images.

To display an image:

```
python_logo.png
```

```
import tkinter as tk

root = tk.Tk()

image = tk.PhotoImage(file="python_logo.png")
label = tk.Label(root, image=image)
label.pack()

root.mainloop()
```

While the layout issue is fixed, you might notice that the image appears heavily enlarged or pixelated on your screen. This is completely normal and not a bug; Tkinter is simply rendering the image in its exact, native pixel dimensions, which often look much larger on modern high-resolution displays.

To gain precise control over these dimensions, we will introduce the **Pillow** library in the next section. Pillow is the industry-standard imaging library for Python, offering advanced tools to resize, crop, and optimize images while integrating seamlessly with Tkinter.



This allows Tkinter applications to become much more visually appealing (*at least we know that it can display an image*).

2.8. Configuration and Tables

Sometimes information should only appear after a user performs an action. This is where `config()` becomes useful.

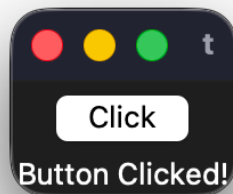
```
import tkinter as tk

root = tk.Tk()

def show_message():
    label = tk.Label(root, text="Hello World")
    label.config(text="Button Clicked!")
    label.pack()

button = tk.Button(root, text="Click", command=show_message)
button.pack()

root.mainloop()
```



The label remains unchanged until the button is pressed.

Tkinter's `Ttreeview` widget allows data to be displayed in a table format.

```
import tkinter as tk
from tkinter import ttk
```

```

root = tk.Tk()
root.title("Sample Table")

# Create the Treeview widget
table = ttk.Treeview(root)
table["columns"] = ("Name", "Age")

# Format the columns
table.column("#0", width=0, stretch=tk.NO) # Hide the default first phantom column
table.column("Name", width=150)
table.column("Age", width=100)

# Create headings
table.heading("Name", text="Name")
table.heading("Age", text="Age")

# Add sample data (5 entries)
samples = [
    ("Alice Smith", 28),
    ("Bob Jones", 34),
    ("Charlie Brown", 22),
    ("Diana Prince", 29),
    ("Evan Wright", 41)]

for name, age in samples:
    table.insert("", tk.END, values=(name, age))

# Pack the widget and run the application
table.pack(padx=10, pady=10)

root.mainloop()

```



Large tables often use a Scrollbar using `tk.Scrollbar(root)`

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Table with Scrollbar")

# Create a frame to hold the table and scrollbar together
frame = tk.Frame(root)
frame.pack(padx=10, pady=10)

# Create the Treeview widget
table = ttk.Treeview(frame)
table["columns"] = ("Name", "Age")

# Format the columns
table.column("#0", width=0, stretch=tk.NO)
table.column("Name", width=150)
table.column("Age", width=100)

# Create headings
table.heading("Name", text="Name")
table.heading("Age", text="Age")
```

```

# Create and link the scrollbar
scrollbar = tk.Scrollbar(frame, orient="vertical", command=
table.yview)
table.configure(yscrollcommand=scrollbar.set)

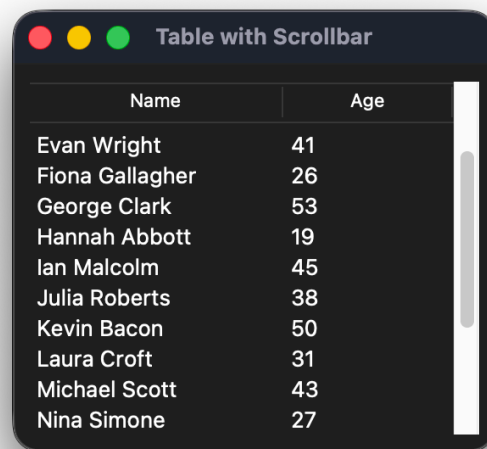
# Pack layout inside the frame (side-by-side)
table.pack(side="left", fill="y")
scrollbar.pack(side="right", fill="y")

# Add 20 sample entries to trigger scrolling
samples = [
    ("Alice Smith", 28), ("Bob Jones", 34), ("Charlie Brow
n", 22),
    ("Diana Prince", 29), ("Evan Wright", 41), ("Fiona Gall
agher", 26),
    ("George Clark", 53), ("Hannah Abbott", 19), ("Ian Malc
olm", 45),
    ("Julia Roberts", 38), ("Kevin Bacon", 50), ("Laura Cro
ft", 31),
    ("Michael Scott", 43), ("Nina Simone", 27), ("Oscar Mar
tinez", 36),
    ("Pam Beesly", 30), ("Quentin Tarantino", 55), ("Rachel
Green", 25),
    ("Sam Winchester", 33), ("Tina Turner", 62)
]

for name, age in samples:
    table.insert("", tk.END, values=(name, age))

root.mainloop()

```



Together, Treeview and Scrollbar allow Tkinter applications to display structured datasets.

2.9. Pop-Up Windows

Sometimes one window isn't enough. Tkinter allows us to create additional windows using `Toplevel()`.

```
import tkinter as tk

root = tk.Tk()
# Display something in the first window
label = tk.Label(root, text="Hello World")
label.pack()

popup = tk.Toplevel(root)
# Display something in the pop-up window
popup_label = tk.Label(popup, text="Hello World Again")
popup_label.pack()

root.mainloop()
```



The first window



The second window

A common use case is Settings windows; Login forms; About dialogs; Help pages.

2.10. Message Boxes

Now let's look at one of the most commonly used GUI features: **Message boxes**. These are small pop-up dialogs used to communicate with users.

- `messagebox.showinfo()` : Displays a pop-up window with a blue information icon and a neutral sound to confirm successful events or share non-urgent details with the user.
- `messagebox.showwarning()` : Opens an alert dialog with a yellow exclamation icon to caution the user about potential risks, incorrect entries, or operations that require careful attention.
- `messagebox.showerror()` : Triggers a critical notification window with a red error icon and an alert sound to inform the user that an operation has completely failed or run into an invalid condition.

Example:

```
import tkinter as tk
from tkinter import messagebox

def calculate_division():
    # Retrieve inputs from entry boxes
    num1_str = entry_num1.get()
    num2_str = entry_num2.get()
```

```

    # Scenario 1: Check for empty input or non-numeric characters
    try:
        val1 = float(num1_str)
        val2 = float(num2_str)
    except ValueError:
        messagebox.showerror("Error", "Invalid input! Please enter numbers only.")
        return

    # Scenario 2: Check for division by zero
    if val2 == 0:
        messagebox.showwarning("Warning", "Cannot divide by zero!")
        return

    # Scenario 3: Successful calculation
    result = val1 / val2
    messagebox.showinfo("Information",
                        f"Calculation successful!\n\n{val1} / {val2} = {result}")

# Set up main window
root = tk.Tk()
root.title("Division Calculator")
root.geometry("300x180")

# Number 1 Input Layout
tk.Label(root, text="Enter First Number:").pack(pady=(10, 0))
entry_num1 = tk.Entry(root)
entry_num1.pack(pady=5)

# Number 2 Input Layout
tk.Label(root, text="Enter Second Number:").pack(pady=(5, 0))

```



```

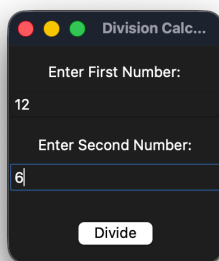
entry_num2 = tk.Entry(root)
entry_num2.pack(pady=5)

# Calculate Button
btn_calculate = tk.Button(root, text="Divide", command=calculate_division)
btn_calculate.pack(pady=15)

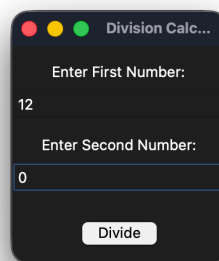
root.mainloop()

```

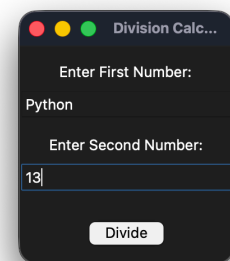
Example output:



The `messagebox.showinfo()` is triggered for the first run since the above entries are valid



The `messagebox.showwarning` is triggered for the second run since the second entry receives 0



The `messagebox.showerror()` is triggered for the third run

Message boxes are simple, but they provide immediate feedback to users. And in GUI applications, feedback is extremely important.

Users should always know: What happened? Whether an operation succeeded? Whether something went wrong? What action they should take

next?

That's exactly what message boxes are designed to do.

2.11. Colors and Sizes

By default, Tkinter applications are fairly plain. They use the operating system's default colors, default window size, and default widget appearance.

Fortunately, Tkinter allows us to customize all of these. A little styling can make an application look much more professional and easier to use.

Some of the most common customizations include: Text colors; Background colors; Window size; Widget size; Whether the window can be resized

Let's look at each of these.

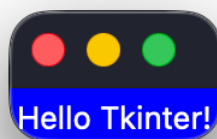
- **Changing Colors:** Tkinter uses two common parameters for colors: `fg` (foreground color) or `bg` (background color)

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="Hello Tkinter!",
                  fg="white", bg="blue")
label.pack()

root.mainloop()
```



In this example, the text is white, and the background is blue

- **Using Named Colors:** Tkinter supports many predefined color names.

Some example colors: `bg=["red", "green", "blue", "yellow", "orange", "purple", "pink", "cyan", "white", "black"]`

These are often sufficient for small projects.

- **Using Hexadecimal Colors:** For more precise control, Tkinter supports hexadecimal color codes just like HTML and CSS. For example: `bg="#3498DB"` or `fg="#FFFFFF"`

This gives access to millions of colors.

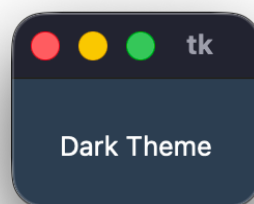
```
import tkinter as tk

root = tk.Tk()

root.configure(bg="#2C3E50")

label = tk.Label(root, text="Dark Theme", fg="white", bg="#2C3E50")
label.pack(padx=20, pady=20)

root.mainloop()
```



Many modern applications use hexadecimal colors to create consistent themes.

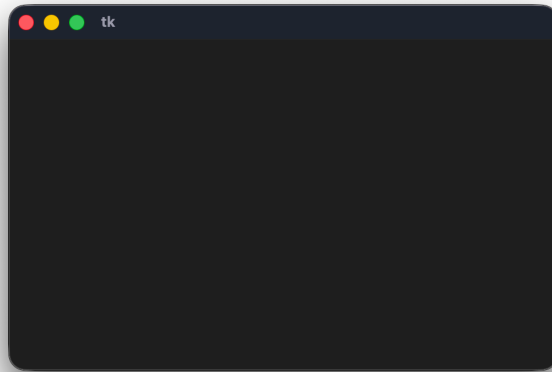
- **Setting Window Size:** By default, Tkinter determines a window size automatically. However, we can specify the exact dimensions using `geometry()`.

```
import tkinter as tk

root = tk.Tk()
root.geometry("500x300")

root.mainloop()
```

The format is ***width x height***. For example: `root.geometry("800x600")` creates a window that is 800 pixels wide x 600 pixels tall



- **Setting Window Position:** We can also specify where the window appears on the screen. For instance: `root.geometry("500x300+200+100")`. This means:
 - Width = 500
 - Height = 300
 - X Position = 200
 - Y Position = 100

The window will appear slightly away from the top-left corner of the screen.

- **Controlling Resizing:** Most operating systems allow users to resize windows by dragging the edges. Sometimes this is useful. Sometimes it completely breaks the layout. Tkinter provides `resizable()` to control this behavior.

Syntax: `root.resizable(False, False)`

```
root.resizable(False, False)
```

The first parameter controls horizontal resizing. The second parameter controls vertical resizing.

Setting	Meaning
<code>False, False</code>	Cannot resize
<code>True, False</code>	Horizontal only
<code>False, True</code>	Vertical only
<code>True, True</code>	Both directions

Example:

```
import tkinter as tk

root = tk.Tk()

root.geometry("400x200")
root.resizable(False, False)

root.mainloop()
```

The output is the same as the previous code but whether you can resize the window is another story

This creates a fixed-size window. Many calculators, login forms, and utility applications use this approach because their layout is designed for a specific size.

- **Adjusting Widget Sizes:** Individual widgets can also have their own sizes. The larger the values, the larger the widget becomes.

```
import tkinter as tk

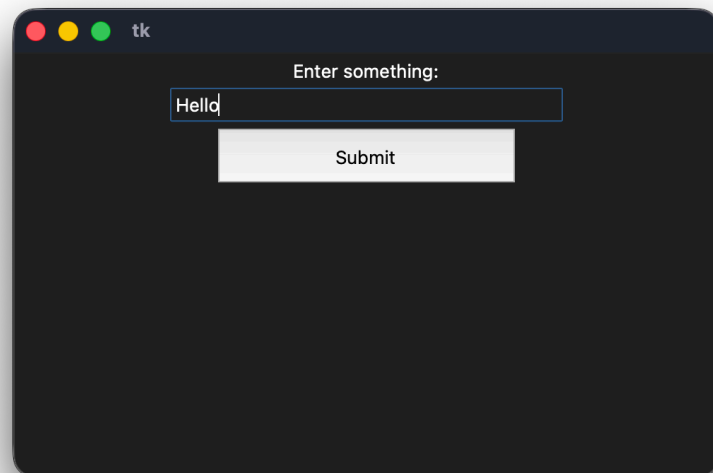
root = tk.Tk()
root.geometry("500x300")

label = tk.Label(root, text="Enter something:")
label.pack()

entry = tk.Entry(root, width=30)
entry.pack()

button = tk.Button(root, text="Submit", width=20, height=2)
button.pack()

root.mainloop()
```



This makes the text field wider.

Colors make an application easier to read. Sizes make an application easier to use. Together, they form the foundation of GUI styling and are often the first things developers customize when building a Tkinter application.



Besides colors and sizes, Tkinter also allows us to customize fonts. The `font` parameter typically follows the format:

```
font=("Arial", 16, "bold")
```

which means: **Font Family = Arial; Font Size = 16; Font Style = Bold**
Example:

```
label = tk.Label(root, text="Welcome", font=("Arial", 18, "bold"))
```

While colors affect appearance, font sizes affect readability. A well-chosen font can make a GUI feel much more professional and user-friendly.

2.12. Read-Only Entries

Sometimes users should be able to see information without modifying it. Examples include: Generated IDs; Calculated results; Usernames; System information; Conversion outputs

Tkinter allows Entry widgets to be placed into a **read-only state**.

```
import tkinter as tk

root = tk.Tk()

entry = tk.Entry(root, state="readonly")
entry.pack()

root.mainloop()
```

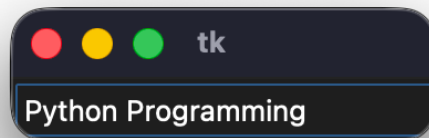
The code above just returns an empty window. Of course, a read-only box isn't very useful if it starts empty.

```
import tkinter as tk

root = tk.Tk()

entry = tk.Entry(root)
entry.insert(0, "Python Programming")
entry.config(state="readonly")
entry.pack()

root.mainloop()
```



The user can select the text, copy the text. But once it becomes read-only, they cannot modify it.

Practical Example: Decimal Converter

A common use case is displaying calculated results.

```
import tkinter as tk

def convert():
    value = int(decimal_entry.get())

    binary_entry.config(state="normal")
    binary_entry.delete(0, tk.END)
    binary_entry.insert(0, bin(value))
    binary_entry.config(state="readonly")
```



```
root = tk.Tk()

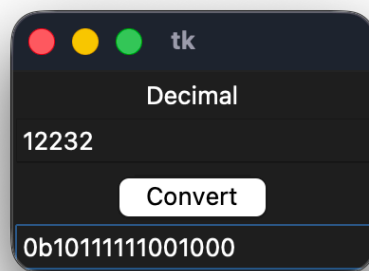
tk.Label(root, text="Decimal").pack()

decimal_entry = tk.Entry(root)
decimal_entry.pack()

tk.Button(root, text="Convert", command=convert).pack()

binary_entry = tk.Entry(root)
binary_entry.pack()

root.mainloop()
```



In this application:

1. The user enters a decimal number.
2. The program calculates the binary value.
3. The result is displayed.
4. The result cannot be accidentally edited.

This is a very common pattern in professional GUI applications.

Read-Only vs Disabled

Tkinter provides another state called `disabled`.

```
entry.config(state="disabled")
```

The difference is:

State	Can View	Can Select	Can Edit
readonly	Yes	Yes	No
disabled	Yes	Usually No	No

For displaying generated results, `readonly` is usually the better choice because users can still copy the content.

3. Some Basic Systems with Tkinter

The previous example demonstrated the fundamental structure of a Tkinter application: creating a window, displaying widgets, handling user input, and responding to events.

Once these foundations are understood, Tkinter can be used to create much larger applications. Many desktop systems are built using the same principles:

- Display information using widgets
- Accept user input
- Process data
- Update the interface dynamically

Below is a simple, complete, and functional Python Tkinter **program** that **creates a desktop window containing a text label, a text entry box, and an interactive click button**

```
import tkinter as tk

def on_click():
    user_name = entry.get()
    label.config(text=f"Hello, {user_name}!")

root = tk.Tk()
root.title("Simple App")
root.geometry("300x150")
```

```

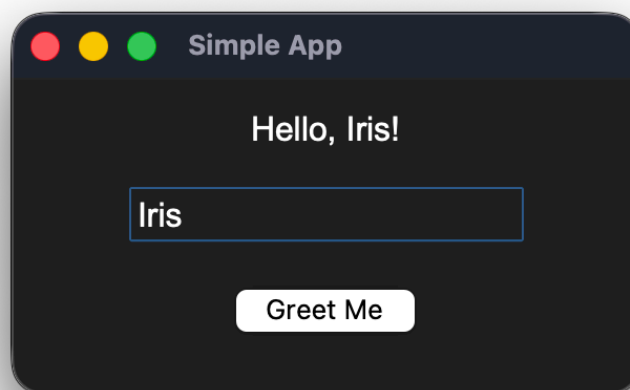
label = tk.Label(root, text="Enter your name:", font=("Arial", 12))
label.pack(pady=10)

entry = tk.Entry(root, font=("Arial", 12))
entry.pack(pady=5)

button = tk.Button(root, text="Greet Me", command=on_click,
bg="lightblue")
button.pack(pady=10)

root.mainloop()

```



- `tk.Tk()` initializes the **main application window container**.
- `root.title()` and `root.geometry()` customize the **window's top title bar** and its **pixel dimensions** (width x height).
- `tk.Label`, `tk.Entry`, and `tk.Button` generate individual structural **graphical interface elements (widgets)**.
- `.pack(pady=10)` serves as the layout manager that positions widgets sequentially on top of each other, using `pady` to append vertical space separation padding.

- `command=on_click` binds the user interaction mouse-click event directly to your custom Python backend logic function.
- `root.mainloop()` is the vital infinite runtime instruction that **keeps the graphical window open and active** on your operating system screen

The simple greeting application introduced the core ideas of Tkinter. These larger projects apply the same concepts on a broader scale, demonstrating how desktop software can be developed entirely in Python.

The following examples show some common systems that can be created using Tkinter.

3.1. Student Information Table

One practical use of Tkinter is displaying tabular data. A simple student management system might store: Student ID; Student Name; Major. While also enable user to insert new students.

Tkinter's `Treeview` widget provides a convenient way to display information in rows and columns.

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox

root = tk.Tk()
root.title("Student Table")
root.geometry("600x550")

# Function to validate and add data to the Treeview
def add_student():
    student_id = ent_id.get().strip()
    name = ent_name.get().strip()
    major = ent_major.get().strip()

    # Validation: Ensure fields are not empty
    if not student_id or not name or not major:
        messagebox.showwarning("Input Error", "All fields m
```

```

ust be filled out!")
    return

    # Insert values into the table
    table.insert("", tk.END, values=(student_id, name, major))

    # Clear fields for the next input
    ent_id.delete(0, tk.END)
    ent_name.delete(0, tk.END)
    ent_major.delete(0, tk.END)
    ent_id.focus()

# --- Form Layout Panel (Inputs using pack) ---
frm_form = tk.LabelFrame(root, text=" Add New Student ", padx=10, pady=10)
frm_form.pack(fill="x", padx=10, pady=10)

# Student ID Row (Side-by-side inside an independent frame)
row_id = tk.Frame(frm_form)
row_id.pack(fill="x", pady=2)
lbl_id = tk.Label(row_id, text="Student ID:", width=12, anchor="w")
lbl_id.pack(side="left")
ent_id = tk.Entry(row_id)
ent_id.pack(side="left", fill="x", expand=True)

# Name Row
row_name = tk.Frame(frm_form)
row_name.pack(fill="x", pady=2)
lbl_name = tk.Label(row_name, text="Name:", width=12, anchor="w")
lbl_name.pack(side="left")
ent_name = tk.Entry(row_name)
ent_name.pack(side="left", fill="x", expand=True)

# Major Row

```

```

row_major = tk.Frame(frm_form)
row_major.pack(fill="x", pady=2)
lbl_major = tk.Label(row_major, text="Major:", width=12, anchor="w")
lbl_major.pack(side="left")
ent_major = tk.Entry(row_major)
ent_major.pack(side="left", fill="x", expand=True)

# Action Button
btn_add = tk.Button(frm_form, text="Add", command=add_student, bg="#4CAF50", fg="blue")
btn_add.pack(fill="x", pady=10)

# --- Table Layout Panel (using pack) ---
table = ttk.Treeview(root, columns=("ID", "Name", "Major"), show="headings")

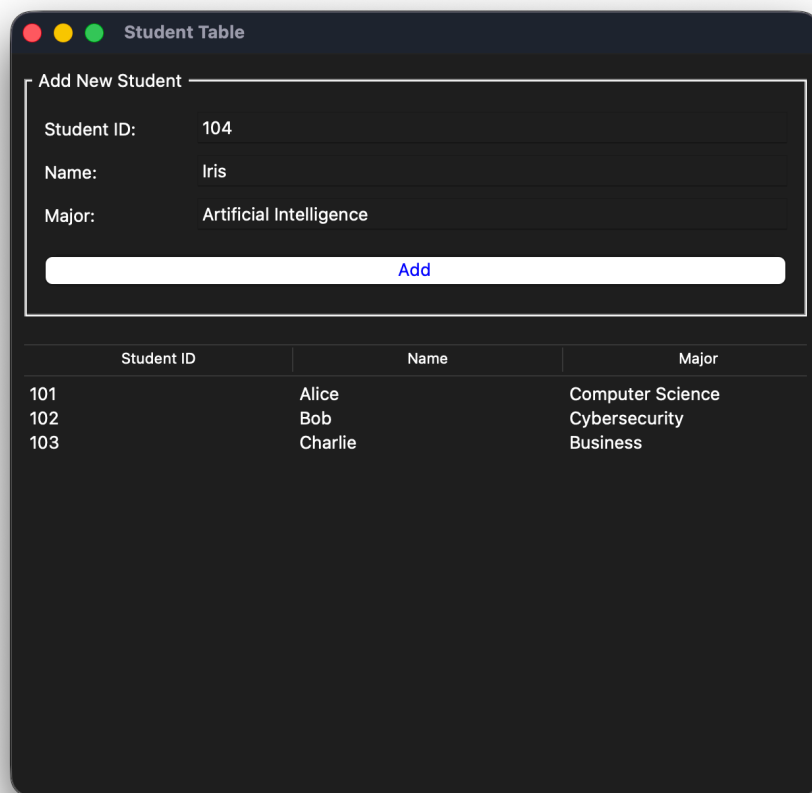
table.heading("ID", text="Student ID")
table.heading("Name", text="Name")
table.heading("Major", text="Major")

# Pre-populated Data
table.insert("", tk.END, values=("101", "Alice", "Computer Science"))
table.insert("", tk.END, values=("102", "Bob", "Cybersecurity"))
table.insert("", tk.END, values=("103", "Charlie", "Business"))

table.pack(fill="both", expand=True, padx=10, pady=10)

root.mainloop()

```



You can also improve this program to validate the Student ID input ("no duplicate IDs" or "must follow a specific pattern")

This simple program creates a table capable of displaying student records.

3.2. Simple Text Editor

A text editor allows users to create, edit, and save text files. It introduces larger widgets and file operations.

```
import tkinter as tk
from tkinter import filedialog
from tkinter import messagebox

window = tk.Tk()
window.title("Simple Text Editor")

window.rowconfigure(0, minsize=800, weight=1)
```

```

window.columnconfigure(1, minsize=800, weight=1)

# File operations logic
def open_file():
    # Open file dialog to choose a .txt file
    filepath = filedialog.askopenfilename(
        filetypes=[("Text Files", "*.txt"), ("All Files",
        "*,*")]
    )
    if not filepath:
        return

    # Clear current content and insert new file content
    txt_edit.delete("1.0", tk.END)
    with open(filepath, "r", encoding="utf-8") as input_file:
        text = input_file.read()
        txt_edit.insert(tk.END, text)

    window.title(f"Simple Text Editor - {filepath}")

def save_file():
    # Open save file dialog with default .txt extension
    filepath = filedialog.asksaveasfilename(
        defaultextension="txt",
        filetypes=[("Text Files", "*.txt"), ("All Files",
        "*,*")]
    )
    if not filepath:
        return

    # Write text component data into the selected file destination
    with open(filepath, "w", encoding="utf-8") as output_file:
        text = txt_edit.get("1.0", tk.END)
        output_file.write(text)

```



```

window.title(f"Simple Text Editor - {filepath}")

txt_edit = tk.Text(window)
frm_buttons = tk.Frame(window, relief=tk.RAISED, bd=2)

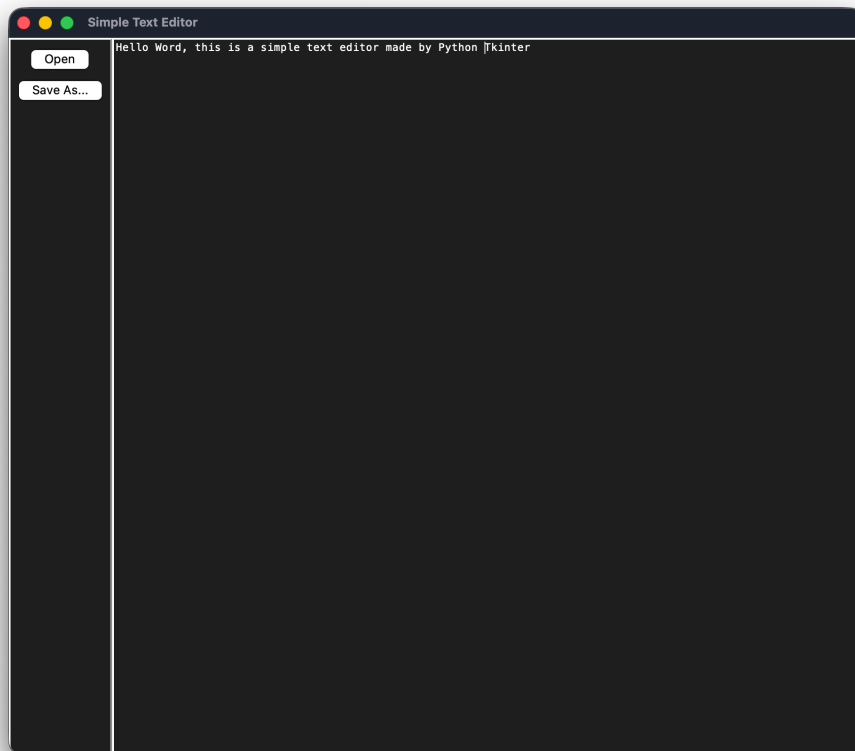
# Connected buttons to functions via the command parameter
btn_open = tk.Button(frm_buttons, text="Open", command=open_file)
btn_save = tk.Button(frm_buttons, text="Save As...", command=save_file)

btn_open.grid(row=0, column=0, padx=5, pady=5)
btn_save.grid(row=1, column=0, padx=5)

frm_buttons.grid(row=0, column=0, sticky="ns")
txt_edit.grid(row=0, column=1, sticky="nsew")

window.mainloop()

```



When you press the Save As button, you can save the note as a `.txt` file

This forms the basis of applications similar to Notepad.

3.3. Drawing Pad

Tkinter provides the `Canvas` widget, which allows users to draw shapes, images, and freehand sketches. A drawing application may include pen tool, brush sizes, color selection, fill color support, eraser tool

```
import tkinter as tk
from tkinter import colorchooser

root = tk.Tk()
root.title("Simple Drawing Pad")

# Track current tools and sizes using a dictionary
settings = {"color": "black",
            "size": 5,
            "prev_x": None,
            "prev_y": None}

# Functions for tools
def choose_color():
    color = colorchooser.askcolor(title="Select Brush Color")
    if color:
        settings["color"] = color

def use_eraser():
    settings["color"] = "white"

def change_size(val):
    settings["size"] = int(val)
```

```

def clear_canvas():
    canvas.delete("all")

# Drawing functions
def start_draw(event):
    settings["prev_x"] = event.x
    settings["prev_y"] = event.y

def draw(event):
    if settings["prev_x"] and settings["prev_y"]:
        # Draw smooth lines instead of disconnected dots
        canvas.create_line(settings["prev_x"], settings["prev_y"], event.x, event.y,
                           fill=settings["color"], width=settings["size"],
                           capstyle=tk.ROUND, smooth=tk.TRUE)
    settings["prev_x"] = event.x
    settings["prev_y"] = event.y

def reset_draw(event):
    settings["prev_x"] = None
    settings["prev_y"] = None

# Toolbar layout
toolbar = tk.Frame(root)
toolbar.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

# Control items
color_btn = tk.Button(toolbar, text="Brush Color", command=choose_color)
color_btn.pack(side=tk.LEFT, padx=5)

eraser_btn = tk.Button(toolbar, text="Eraser", command=use_

```

```

eraser)
eraser_btn.pack(side=tk.LEFT, padx=5)

clear_btn = tk.Button(toolbar, text="Clear All", command=clear_canvas)
clear_btn.pack(side=tk.LEFT, padx=5)

size_label = tk.Label(toolbar, text=" Size:")
size_label.pack(side=tk.LEFT)

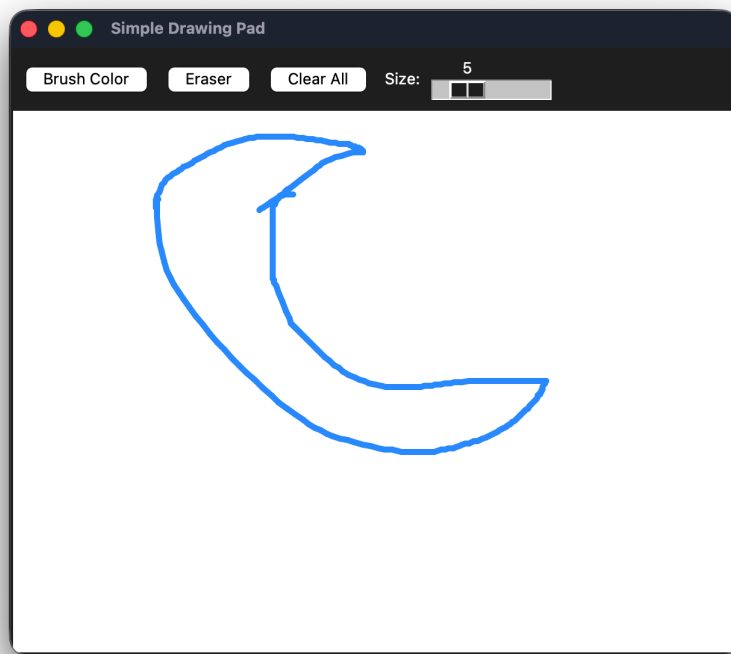
size_slider = tk.Scale(toolbar, from_=1, to=20, orient=tk.HORIZONTAL, command=change_size)
size_slider.set(settings["size"])
size_slider.pack(side=tk.LEFT, padx=5)

# Canvas setup
canvas = tk.Canvas(root, bg="white", width=600, height=450)
canvas.pack(fill=tk.BOTH, expand=True)

# Event bindings
canvas.bind("<Button-1>", start_draw)
canvas.bind("<B1-Motion>", draw)
canvas.bind("<ButtonRelease-1>", reset_draw)

root.mainloop()

```



This program allows the user to draw by dragging the mouse.

3.4. Simple Calculator

Calculators are excellent beginner projects because they combine user input, mathematical operations, and dynamic output

```
import tkinter as tk
from tkinter import messagebox

def press(value):
    display.insert(tk.END, value)

def clear():
    display.delete(0, tk.END)

def negative():
```

```

text = display.get()

if text.startswith("-"):
    display.delete(0)
else:
    display.insert(0, "-")

def calculate():
    try:
        result = eval(display.get())
        display.delete(0, tk.END)

        if isinstance(result, float) and result.is_integer
():
            result = int(result)

        display.insert(0, str(result))

    except:
        messagebox.showerror("Error", "Invalid expression")

root = tk.Tk()
root.title("Calculator")
root.resizable(False, False)

# Large Display
display = tk.Entry(root, font=("Arial", 20), justify="right", bd=5)
display.pack(fill="x", padx=10, pady=10, ipady=10)

main_frame = tk.Frame(root)
main_frame.pack(pady=5)

# Left Side
left_frame = tk.Frame(main_frame)
left_frame.pack(side="left", padx=5)

```

```

# Operators row
op_frame = tk.Frame(left_frame)
op_frame.pack()

for op in ["+", "-", "*", "/"]:
    tk.Button(op_frame, text=op, width=5, height=2, font=
("Arial", 12),
              command=lambda o=op: press(o)).pack(side="left",
padx=2)

# Utility buttons under operators
util_frame = tk.Frame(left_frame)
util_frame.pack(pady=5)

tk.Button(util_frame, text="+/-", width=10, height=2,
          command=negative).pack(side="left", padx=2)
tk.Button(util_frame, text=".", width=10, height=2,
          command=lambda: press(".")).pack(side="left", pad
x=2)

# Number Pad
num_frame = tk.Frame(left_frame)
num_frame.pack(pady=10)

numbers = [
    ["7", "8", "9"],
    ["4", "5", "6"],
    ["1", "2", "3"]]

for r, row in enumerate(numbers):
    for c, num in enumerate(row):
        tk.Button(num_frame, text=num, width=6, height=2, f
ont=("Arial", 12),
                  command=lambda n=num: press(n)).grid(row=
r, column=c, padx=2, pady=2)

```

```

tk.Button(num_frame, text="0", width=6, height=2, font=("Ar
ial", 12),
          command=lambda: press("0")).grid(row=3, column=1,
padx=2, pady=2)

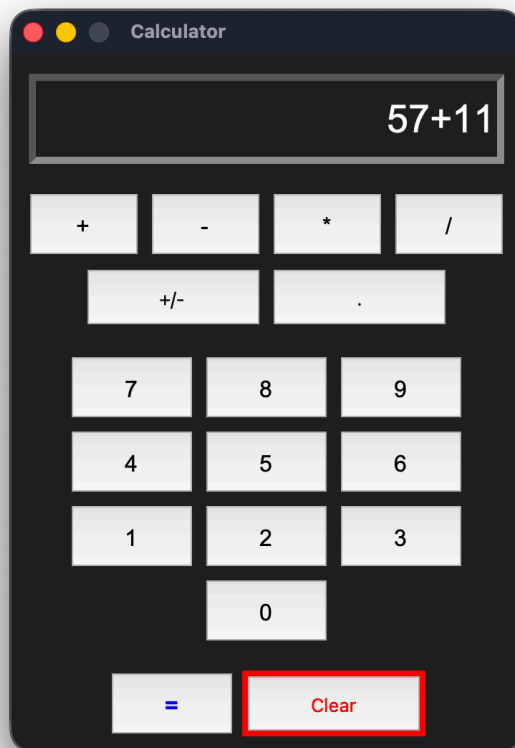
# Bottom row (=, Clear)
bottom_frame = tk.Frame(left_frame)
bottom_frame.pack(pady=5)

tk.Button(bottom_frame, text="=", width=6, height=2, font=
("Arial", 12, "bold"), fg="blue",
          command=calculate).pack(side="left", padx=2)

tk.Button(bottom_frame, text="Clear", width=10, height=2, f
g="red",
          highlightbackground="red", highlightthickness=2,
          command=clear).pack(side="left", padx=2)

root.mainloop()

```

3.5. Tic-Tac-Toe Game

Games are a fun way to learn event-driven programming. A Tic-Tac-Toe game requires a 3×3 grid, button click events, turn tracking, win detection, game reset functionality

```
# Basic tic-tac-toe game
from tkinter import *
import random

# Global variables
characters = ["x", "o"]
player = random.choice(characters)
buttons = [[None for _ in range(3)] for _ in range(3)]
label = None

def restart():
    global player
```

```

player = random.choice(characters)
label.config(text=player + " 's turn", fg="white")
for row in range(3):
    for column in range(3):
        buttons[row][column].config(text="", fg="black", highlightbackground="black", highlightthickness=0)

def next_turn(row, column):
    global player

    if buttons[row][column]['text'] == "" and check_winner() [0] is False:
        buttons[row][column]['text'] = player
        buttons[row][column]['fg'] = "blue" if player == "x" else "red"
        result, winning_cells = check_winner()
        if result is False:
            player = characters[0] if player == characters[1] else characters[1]
            label.config(text=player + " 's turn", fg="white")
        elif result is True:
            label.config(text=player + " wins!", fg="blue" if player == "x" else "red")
            # Highlight winning buttons
            color = "blue" if player == "x" else "red"
            for r, c in winning_cells:
                buttons[r][c].config(highlightbackground=color, highlightthickness=4)
        elif result == "Draw":
            label.config(text="Draw!", fg="yellow")

def check_winner():
    # Check rows
    for row in range(3):
        if buttons[row][0]['text'] == buttons[row][1]['text'] == buttons[row][2]['text'] and buttons[row][0]['text'] != "":
            return True, [row, 0, 1, 2]
        elif buttons[row][0]['text'] == buttons[row][1]['text'] == buttons[row][2]['text'] and buttons[row][0]['text'] != "":
            return True, [row, 0, 1, 2]
        elif buttons[0][row] == buttons[1][row] == buttons[2][row] and buttons[0][row] != "":
            return True, [0, 1, 2, row]
        elif buttons[0][row] == buttons[1][row] == buttons[2][row] and buttons[0][row] != "":
            return True, [0, 1, 2, row]
    # Check diagonals
    if buttons[0][0] == buttons[1][1] == buttons[2][2] and buttons[0][0] != "":
        return True, [0, 1, 2, 0, 1, 2]
    if buttons[0][2] == buttons[1][1] == buttons[2][0] and buttons[0][2] != "":
        return True, [0, 1, 2, 2, 1, 0]
    return False, []

```

```

t'] == buttons[row][2]['text'] != '':
    return True, [(row, 0), (row, 1), (row, 2)]
# Check columns
for column in range(3):
    if buttons[0][column]['text'] == buttons[1][column]
['text'] == buttons[2][column]['text'] != '':
        return True, [(0, column), (1, column), (2, col
umn)]
# Diagonals
if buttons[0][0]['text'] == buttons[1][1]['text'] == bu
ttons[2][2]['text'] != '':
    return True, [(0, 0), (1, 1), (2, 2)]
if buttons[0][2]['text'] == buttons[1][1]['text'] == bu
ttons[2][0]['text'] != '':
    return True, [(0, 2), (1, 1), (2, 0)]
# Draw
if empty_spaces() is False:
    return "Draw", []
return False, []

def empty_spaces():
    for row in range(3):
        for column in range(3):
            if buttons[row][column]['text'] == '':
                return True
    return False

def make_move(r, c):
    return lambda: next_turn(r, c)

def main():
    global label, buttons

    board = Tk()
    board.title("Tic-Tac-Toe")

```

```

    label = Label(text=player + " 's turn", font=("helvetica", 40))
    label.pack(side="top")

    new_game_button = Button(text="New Game",
                             font=("helvetica", 20), command=restart)
    new_game_button.pack(side="top")

    frame = Frame(board)
    frame.pack()

    for row in range(3):
        for column in range(3):
            buttons[row][column] = Button(frame, text="", font=('consolas', 40), width=5, height=2,
                                           command=make_move(row, column))
            buttons[row][column].grid(row=row, column=column)

    board.mainloop()

main()

```



3.6. Other Systems Built with Tkinter

Once comfortable with widgets and event handling, Tkinter can be used to create many useful desktop applications:

- **Educational Systems:** Quiz applications; Flashcard systems; Language learning tools; Mathematics practice programs; Student management systems
- **Productivity Applications:** To-do lists; Note-taking software; Budget trackers; Contact managers; Inventory systems
- **Media Applications:** Music players; Video launchers; Image viewers; Slideshow applications
- **Internet-Based Tools** . Tkinter can also work with online services and APIs like YouTube video search tools; Weather applications; Currency converters; News readers; Chat applications

Summary

This section introduced the fundamentals of Tkinter, Python's built-in library for creating graphical user interfaces (GUIs).

We explored how to create windows and work with many of the widgets that form the foundation of desktop applications, including:

- Labels and Frames
- LabelFrames
- Entry and Text widgets
- Buttons and selection widgets
- User input processing
- Layout managers (`pack` , `grid` , and `place`)
- Canvas and image display
- Tables and Treeviews
- Pop-up windows
- Message boxes
- Colors, fonts, and window sizing
- Read-only fields for displaying generated results

By this point, you should have a general understanding of how Tkinter applications are structured and how different widgets work together to create an interactive user experience.

However, it's important to remember that this chapter only scratches the surface. Tkinter is one of those libraries where practical experience matters far more than memorization. Reading about widgets is helpful, but real understanding comes from building applications and experimenting with them yourself.

If you want to become comfortable with Tkinter, consider:

- Reading Python GUI books and documentation
- Watching Tkinter tutorials and project walkthroughs
- Building small desktop applications
- Recreating software you already use
- Using AI assistants to help explain errors and generate project ideas

Even simple projects such as a calculator, login form, unit converter, notepad, or inventory system can teach a tremendous amount about GUI programming.

And while Tkinter is responsible for creating windows, buttons, text boxes, and layouts, many modern applications also need to work with images.

Photo viewers, image editors, OCR systems, machine learning tools, computer vision projects, and media applications all require image-processing capabilities that Tkinter alone does not provide.

This is where our next library comes in. In the next section, we will explore **Pillow (PIL)**, Python's most popular image-processing library. Pillow allows us to:

- Open images
- Resize images
- Crop images
- Rotate images
- Apply filters
- Manipulate pixels
- Save images in different formats

And perhaps most importantly: **Tkinter and Pillow are frequently used together**. Tkinter provides the graphical interface. Pillow provides the image-processing functionality.

Together, they form one of the most common combinations for building desktop applications that work with images.

So now that we've learned how to build the window, let's learn how to put pictures inside it—and manipulate them with code.

References

<https://docs.python.org/3/library/tkinter.html>

<https://www.geeksforgeeks.org/python/python-gui-tkinter/>

<https://realpython.com/python-gui-tkinter/>

Next Section:

 [29. Python: Image Processing](#)